

Java---SSH(MVC)

1. 谈谈你 mvc 的理解

MVC 是 Model—View—Controler 的简称。即模型—视图—控制器。MVC 是一种设计模式，它强制性的把应用程序的输入、处理和输出分开。

MVC 中的模型、视图、控制器它们分别担负着不同的任务。

视图: 视图是用户看到并与之交互的界面。视图向用户显示相关的数据，并接受用户的输入。视图不进行任何业务逻辑处理。

模型: 模型表示业务数据和业务处理。相当于 JavaBean。一个模型能为多个视图提供数据。这提高了应用程序的重用性

控制器: 当用户单击 Web 页面中的提交按钮时,控制器接受请求并调用相应的模型去处理请求。

然后根据处理的结果调用相应的视图来显示处理的结果。

MVC 的处理过程: 首先控制器接受用户的请求, 调用相应的模型来进行业务处理, 并返回数据给控制器。控制器调用相应的视图来显示处理的结果。并通过视图呈现给用户。

Struts

1. struts1.2 和 struts2.0 的区别? 如何控制两种框架中的单例模式?

struts1.2 和 struts2.0 的对比

a、Action 类:

struts1.2 要求 Action 类继承一个基类。struts2.0 Action 要求继承 ActionSupport 基类

b、线程模式

struts1.2 Action 是单例模式的并且必须是线程安全的,因为仅有一个 Action 的实例来处理所有的请求。

单例策略限制了 Struts1.2 Action 能做的事情,并且开发时特别小心。Action 资源必须是线程安全的或同步的。

struts2.0 Action 为每一个请求产生一个实例,因此没有线程安全问题。

c、Servlet 依赖

struts1.2 Action 依赖于 Servlet API, 因为当一个 Action 被调用时 HttpServletRequest 和 HttpServletResponse 被传递给 execut 方法。

struts2.0 Action 不依赖于容器,允许 Action 脱离容器单独测试。如果需要,Struts2 Action 仍然可以访问初始的 Request 和 Response。

但是,其他的元素减少或者消除了直接访问 HttpServletRequest 和 HttpServletResponse 的必要性。

d、可测性

测试 struts1.2 Action 的一个主要问题是 execute 方法暴露了 Servlet API(这使得测试要依赖于容器)。

一个第三方扩展: struts TestCase

提供了一套 struts1.2 的模拟对象来进行测试。

Struts2.0 Action 可以通过初始化、设置属性、调用方法来测试,“依赖注入”也使得测试更容易。

2. 项目中为什么使用 SSH

1. 使用 Struts 是因为 struts 是基于 MVC 模式的,很好的将应用程序进行了分层,使开发者更关注于业务逻辑的实现;第二, struts 有着丰富的 taglib,如能灵活运用,则能大大提高开发效率。

2. 使用 Hibernate: 因为 hibernate 为 Java 应用提供了一个易用的、高效率的对象关系映射框架。hibernate 是个轻量级的持久性框架,功能丰富。

3. 使用 Spring: 因为 spring 基于 IoC(Inversion of Control, 反向控制)和 AOP 构架多层 j2ee 系统的框架,但它不强迫你必须在每一层中必须使用 Spring, 因为它模块化的很好, 允许你根据自己的需要选择使用它的某一个模块;

采用 IoC 使得可以很容易的实现 bean 的装配, 提供了简洁的 AOP 并据此实现事务管理(Transaction Managment),等等

3. struts 与 spring 是如何继承的

不懂题目之意: 是否是说的 struts 的 action 都继承自 org.apache.struts.action.Action 而 Spring 中的继承就不清楚了。

4. struts 如何实现国际化

以下以两国语言(中文, 英文)为例:

1. 在工程中加入 Struts 支持

2. 编辑 ApplicationResource.properties 文件, 在其中加入要使用国际化的信息, 例如:

lable.welcome.china=Welcome!!!

3. 创建英文资源文件 ApplicationResource_en.properites

4. 创建临时中文资源文件 ApplicationResource_temp.properites 例如: lable.welcom.china=中国欢迎您!

5. 对临时中文资源文件进行编码转换。可以使用 myeclipse 的插件, 也可以在 dos 下执行:

native2ascii -encoding gb2312 ApplicationResource_temp.properties

ApplicationResource_zh_CN.properties

6. 在 jsp 中加入 struts 的 bean 标记库 <%@ taglib uri="/WEB-INF/struts-bean.tld" prefix="bean"%>

显示信息: <bean:message key="label.welcome.china">

可以在 struts-config.xml 文件中定义多个资源包, 而每个资源包可使用 key 属性指定包的名称。

5. struts2.0 的常用标签

1. 往 action 里传值: <input name="userName" type="text" class="input6" size="15">

2. 显示标签 property 用于输出指定值: <s:property value="userName" />

3. 用于从页面往 action 中(user)的对象内传值: <s:text name="user.userName" id="username"/>

4. 判断<s:if> </s:if> 用于在页面中判断指定数据 <s:if test="userName == admin">... </s:if> <s:else>... </s:else>

5. 迭代<s:iterator>用于将 List、Map、ArrayList 等集合进行循环遍历

<s:iterator value="userList" id="user" status="u">

<s:property value="userName"/>

</s:iterator>

6. URL 地址标签, <s:url>用于生成一个 URL 地址, 可以通过 URL 标签指定的<s:param>子元素向 URL 地址发送请求参数

<s:url action=" ">

<s:param name=" " value=""/></s:param>

</s:url>

7. 超链接 <a href>一般和<s:url>标签一起使用, 用于带多个参数。

<a href="

<s:url action=" ">

<s:param name=" " value=""/></s:param>

<s:param name=" " value=""/></s:param>

<s:param name=" " value=""/></s:param>

</s:url>

">超链接

8. set 标签, 用于将某个值放入指定的范围内。例如 application,session 等。

<s:set name="user" value="userName" scope="request"/>

6. struts 中怎么配置 form-bean、action、tiles

此处配置的是 struts1.2 的

form-bean 配置: (在配置文件 struts-config.xml 中):

```
<form-beans >
```

```
  <form-bean name="" type=""></form-bean>
```

```
</form-beans>
```

name: 指定 form 的名字; type 指定 form 的类型: 包名+类名;

action 配置: (在配置文件 struts-config.xml 中)

```
<action-mappings >
```

```
  <action path=""
```

```
    attribute=""
```

```
    input=""
```

```
    name=""
```

```
    parameter=""
```

```
    scope="request"
```

```
    type=""
```

```
  ></action>
```

```
</action-mappings>
```

path: 请求 Action 的名字; attribute: form 的名字,与 form-bean 中的 name 对应; input: 输入页的路径;

name: 如果配置了 attribute, name 不起作用,与 attribute 一样; parameter: 使用分发 Action 时,指定调用分发 Action 中的方法名;

scope: Action 的范围; type: Action 的类型: 包名+类名;

tiles 配置:

1. 新建一个 tiles-defs.xml 文件,并在其中做如下配置:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE tiles-definitions PUBLIC "-//Apache Software Foundation//DTD Tiles Configuration 1.1//EN"
```

```
"http://jakarta.apache.org/struts/dtds/tiles-config_1_1.dtd">
```

```
<tiles-definitions>
```

```
  <definition name="member-definition" path="/Jsp/layout.jsp">
```

```
    <put name="top" value="/mTop.do"/>
```

```
    <put name="left" value="/mLeft.do"/>
```

```
    <put name="main" value="/defaultMmain.do"/>
```

```
  </definition>
```

```
</tiles-definitions>
```

2. 在 web.xml 和 struts-config.xml 和 web.xml 中做相应配置:

在 struts-config.xml 中配置如下:

```
*****
```

```
<plug-in className="org.apache.struts.tiles.TilesPlugin">
```

```
  <set-property property="definitions-config" value="/WEB-INF/tiles-defs.xml"/>
```

```
  <set-property property="definitions-parser-validate" value="true"/>
```

```
</plug-in>
```

```
*****
```

在 web.xml 中配置如下:

```
<init-param>
  <param-name>definitions-config</param-name>
  <param-value>/WEB-INF/tiles-defs.xml</param-value>
</init-param>
```

3. 调用 Tiles 组件

Tiles 组件是通过 Struts Action 来调用的，在 struts-config.xml 中做如下 action 配置：

```
<action
  path="/member"
  parameter="member-definition"
  type="org.apache.struts.actions.ForwardAction"/>
```

7. logic 标签有哪几个？

此标签库可以分为三种类型：条件、循环、转发/重定向。

1. 条件类型 (没标注的都比较简单)

logic:empty, logic:notEmpty;

logic:equal, logic:notEqual, logic:lessThan, logic:greaterThan, logic:lessEqual, logic:greaterEqual;

logic:present, logic:notPresent;

logic:match, logic:notMatch; 比较两字符串是否相等，可以比较字符串的开始的、结尾的或其中的某一部分。location 属性：指定从某个位置开始进行比较。

2. 循环类型

logic:iterate

3. 转发/重定向类型

logic:forward 和 logic:redirect

logic:forward 标签和 jsp:forward 标签相似，但它可以使用 global forward 中的一个 ActionForward

例如：<logic:forward name="login" />

与上面相关的 global forward 中的代码：

```
<global-forwards>
  <forward name="login" path="/loginForm.jsp"/>
</global-forwards>
```

logic:redirect 标签和上面的标签类似，但它默认调用的方法是 response.sendRedirect(),

取代了上面的 requestDispatcher.forward()。最大的不同是它支持所有 html:link 标签的属性，所以它能够指定 request 参数：

```
<logic:redirect name="login" paramId="employeeId" paramName="employee" property="id" />
```

在 MVC 框架下，不推荐使用这两个标签，你应该是从 controller 中选择下一个 view，而不是从 view 中选择。在 Jsp 页面中不要过多的使用 logic 标签。

8. action 是单实例还是多实例，为什么？

action 是单实例的。当多个用户访问一个请求的时候，服务器内存中只有一个与之对应的 action 类对象。

因为当服务器第一次加载 struts 的配置文件的时候，创建了一个 Action 后，每发送一个请求，服务器都会

先去检索相应的范围内(request,session)是否存在

这样一个 action 实例,如果存在,则使用这个实例,如果不存在,则创建一个 action 实例。

9. dispatchAction 是用什么技术实现的?

DispatchAction 是 Action 的一个子类,主要解决了一个请求处理多个功能的问题

普通的 Action 你只能写 execute 方法来处理业务,而想用这一个 Action 处理多个任务,你必须要请求参数进行解析,用 if 语句块来处理

举一个小例子:

有如下一个 url: <http://localhost:8080/myApp/addUserAction.do>

如果你处理这个 url 的是一个普通的 Action,那么就只能在 execute 里面执行插入的相关操作,如果换成一下 url:

<http://localhost:8080/myApp/UserAction.do?method=add>

你就应该根据 method 对象的值来执行相应的操作,如再有一个路径 <http://localhost:8080/myApp/UserAction.do?method=delete>

这个还可以使用那个 Action 来处理的,只是多判断一下而已。

如果你用 DispatchAction,就简单多了,所以 dispatchAction 还是用的 Action 的这么一个技术。

10. struts2.0 的 mvc 模式? 与 struts1.0 的区别?

struts2 的 mvc 模式: 当用户在页面提交用户请求时,该请求需要提交给 struts2 的控制器处理。struts2 的控制器根据处理结果,

决定将哪个页面呈现给客户端。

与 struts1 最大的不同是: struts2 的控制器。struts2 的控制器不再像 struts1 的控制器,需要继承一个 Action 父类,甚至可以无需实现

任何接口, struts2 的 Action 就是一个普通的 POJO。实际上, Struts2 的 Action 就是一个包含 execute 方法的普通 Java 类

该类里包含的多个属性用于封装用户的请求参数。

11. STRUTS 的配置文件是什么?

struts-config.xml

12. 请写出 Struts 的工作原理。

13. struts 的处理流程。

一、ActionServlet 的初始化

ActionServlet 作为 Struts 组件的前端控制器, 由于 web.xml 的相应配置:

```
<load-on-startup>0</load-on-startup>
```

在应用一加载时即会被实例化并调用其 init 方法, init 方法所做的主要工作有二:

1. 加载 struts 配置文件, 并创建用于封装配置信息的 ModuleConfig 对象
2. 加载资源文件, 并创建用于封装资源文件的 MessageResources 对象

需要注意两点: 如果 web.xml 有多模块配置, 将创建多个 ModuleConfig 对象和 MessageResources 对象分别

用于封装各个模块的 struts 配置文件和资源文件。

针对各个模块所创建的 ModuleConfig 对象和 MessageResources 对象将存储在 ServletContext 中, 对应的属

性名中有该模块名称的相应标识。

另外，如果有相应配置的话，init 方法还将初始化数据源和 PlugIn

二、ActionServlet 的 process

所有形如*.do 的请求（根据 web.xml 中的相关配置）将提交给 ActionServlet，最终将调用其 process 方法

。 process 方法的主要工作有三：

1. 根据请求信息获知所请求的模块名称，从 ServletContext 中获得与请求模块对应的的 ModuleConfig 对象，并存储到 request 中。
2. 根据模块信息获得相应的 RequestProcessor 对象，一个模块对应一个 RequestProcessor 对象，RequestProcessor 对象将关联与所属模块对应的 ModuleConfig 对象。
3. 调用 RequestProcessor 对象的 process 方法，将 request 和 response 作为参数传递给它。

三、RequestProcessor 的 process

RequestProcessor 对象的 process 方法的主要工作有五步：

1. 调用自己的 processPreprocess(request, response)方法，该方法不进行任何操作，用于子类重写扩展其功能。
2. 获得与请求地址所匹配的 ActionMapping 对象，AcionMapping 对象用于封装一个特定 acion 的配置信息。
3. 根据 ActionMapping 中的配置信息获得 ActionForm 对象（该对象将缓存到 request 或 session 中），并将表单数据填充到 ActionForm 中，然后根据 ActionMapping 的配置决定是否验证 ActionForm，如果验证，将调用 ActionForm 的 validate 方法，若其返回的 ActionErros 对象中包含 ActionMessage 对象则表示验证失败，将转向 action 配置信息 input 属性所指示的地址。
4. 如果 ActionForm 无需验证或验证通过将创建并缓存与请求地址匹配的 Action 对象，将 ActionMappin g 对象、ActionForm 对象、request 和 response 作为参数调用其 execute 方法。
5. 根据 Action 对象的 execute 方法返回的 ActionForward 对象，将请求转发或重定向到该 ActionForwa rd 所封装的地址。

14. 用 struts +hibernate 做一个增删查改(机试)

15. Struts2.0 与 Struts1.x 有何区别。

特征	Struts 的一	Struts 的 2
行动班	Struts 的一要求采取行动班，以延长一个抽象的基类。 一个共同的问题在 Struts 的	12Struts 的行动，可能实施一项行动的界面，随着其他接口，使可选和定制服务。 Struts

	一，是编程抽象类而不是接口。	的 2 提供了相应的 actionsupport 一流的执行常用的接口。 虽然，这项行动的界面是 不是必需的 。 任何波霍对象与执行的签名可以被用来作为一个 Struts 的二行动的对象。
线程模型	Struts 的一行动是单身，必须线程安全的，因为将只有一个实例一类来处理所有的请求采取行动。 单身人士策略地方的限制，可以做些什么与 Struts 的一行动，并要求加倍小心发展。 行动的资源，必须线程安全的或同步。	2 Struts 的行动对象是实例对于每个请求，因此没有线程安全问题。 （在实践中， Servlet 的容器，产生许多扔离家出走的对象，每次请求，并多一个对象不施加表现罚款，或影响垃圾收集）。
Servlet 的依赖	Struts 的一行动，依赖于该 Servlet 的空气污染指数以来， HttpServletRequest 和 HttpServletResponse 传递给 Execute 方法，当一个行动是引用。	Struts 的行动，二是不耦合的一个货柜。 最经常的 Servlet 的背景是派代表作为简单的地图，让行动，以测试陷入孤立的境地。 2 Struts 的行动仍然可以使用原来的请求和响应，如果需要。 不过，其他建筑元素，减少或消除需要访问 HttpServletRequest 或 HttpServletResponse 直接。
测试	一个主要障碍，以测试 Struts 的一行动是 Execute 方法暴露了 Servlet 的空气污染指数。 第三党的扩展， Struts 的 testcase ，提供了一套模拟对象的 Struts 1 。	2 Struts 的行动可以测试实例化的行动，设置属性，引用的方法。 依赖注入的支持也使测试更简单。
收获的投入	Struts 的一使用 actionform 对象捕获输入。 一样的行动，所有 actionforms 必须扩大基地，一流的。 由于其他的 JavaBeans 不能被用来作为 actionforms ，开发商常常造成多余的班，以捕捉的投入。 dynabeans 可以用来作为一种替代创造常规 actionform 班，但这里太，发展商可能会 redescribing 现有的 JavaBeans 。	Struts 的 2 使用性能的行动作为输入属性，不再需要第二个输入对象。 输入属性可能是丰富的对象类型可能有自己的属性。 行动性质可以从网页上通过 taglibs 。 Struts 的 2 还支持 actionform 模式，以及波霍形式，对象和波霍行动。 丰富的对象类型，其中包括商业利益或域对象，可以被用来作为输入/输出对象。 该 modeldriven 功能简化 taglb 的提述，波霍投入对象。
语言表达	Struts 的一整合与 jstl ，所以它使用 jstl 下午。 的 EI 已基本对象图遍历，但相对薄弱的收集和索引财产的支持。	Struts 的二可以使用 jstl ，但框架也支持一个更强大和灵活的语言表达所谓的“对象图形符号语言”（ ognl ）。
有约束力的价值观纳入意见	Struts 的一使用标准的 JSP 机制，有约束力的物体进入该网页的背景下访问。	Struts 的 2 使用“ valuestack ”技术，使该 taglibs 可以访问的价值观没有耦合您的看法，以对象类型，这是渲染。 该 valuestack 策略允许重用的意见覆盖的类型可能有相同的属性名称，但不同的属性类型。

类型转换	Struts 的一 actionform 属性通常是所有字符串。 Struts 的一用途的商品-beanutils 为类型转换。 转换器每级的，而不是配置的每个实例。	Struts 的 2 使用 ognl 为类型转换。 该框架包括转换为基本和共同的对象类型和原始。
验证	Struts 的一支持手册审定通过验证方法对 actionform ，或通过一个扩展到商品校验。班级可以有不同的验证背景为同一阶层，但不能链，以验证就分对象。	Struts 的 2 支持手册审定通过验证的方法和 xwork 的验证框架。 该 xwork 的验证框架支持链接验证分为分性能的使用验证的定义为性能一流的类型和验证。
控制行动的 执行	Struts 的一支持独立的请求处理器（生命周期）为每个单元，但所有的行动，在该模块必须共用相同的生命周期。	2 Struts 的支持，创造不同的生命周期对每行动的基础上通过拦截堆叠。 自定义栈，才能创造和使用不同的行动，视需要。

16. Struts/webwork 的工作机制，它有哪些标签

Struts 2 框架的大致处理流程如下：

- 1 浏览器发送请求，例如请求/mypage.action、/reports/myreport.pdf 等。
- 2 核心控制器 FilterDispatcher 根据请求决定调用合适的 Action。
- 3 WebWork 的拦截器链自动对请求应用通用功能，例如 workflow、validation 或文件上传等功能。
- 4 回调 Action 的 execute 方法，该 execute 方法先获取用户请求参数，然后执行某种数据库操作，既可以是将数据保存到数据库，也可以从数据库中检索信息。实际上，因为 Action 只是一个控制器，它会调用业务逻辑组件来处理用户的请求。
- 5 Action 的 execute 方法处理结果信息将被输出到浏览器中，可以是 HTML 页面、图像，也可以是 PDF 文档或者其他文档。此时支持的视图技术非常多，既支持 JSP，也支持 Velocity、FreeMarker 等模板技术。

Hibernate

1. 什么是 Hibernate 的并发机制？怎么去处理并发问题？

Hibernate 并发机制：

- a、Hibernate 的 Session 对象是非线程安全的,对于单个请求,单个会话,单个的工作单元(即单个事务,单个线程),它通常只使用一次,然后就丢弃。
- 如果一个 Session 实例允许共享的话，那些支持并发运行的,例如 Http request,session beans 将会导致出现资源争用。
- 如果在 Http Session 中有 hibernate 的 Session 的话,就可能会出现同步访问 Http Session。只要用户足够快的点击浏览器的“刷新”，就会导致两个并发运行的线程使用同一个 Session。
- b、多个事务并发访问同一块资源,可能会引发第一类丢失更新，脏读，幻读，不可重复读，第二类丢失更新一系列的问题。

解决方案：设置事务隔离级别。

Serializable：串行化。隔离级别最高

Repeatable Read：可重复读

Read Committed：已提交数据读

Read Uncommitted：未提交数据读。隔离级别最差

设置锁：乐观锁和悲观锁。

乐观锁：使用版本号或时间戳来检测更新丢失,在<class>的映射中设置 optimistic-lock="all"可

以在没有版本或者时间戳属性映射的情况下实现 版本检查，此时 Hibernate 将比较一行记录的每个字段的
状态 行级悲观锁：Hibernate 总是使用数据库的锁定机制，从不在内存中锁定对象！只要为 JDBC 连接指定
一下隔 离级别，然后让数据库去搞定一切就够了。类 LockMode 定义了 Hibernate 所需的不同的锁定级别：
LockMode.UPGRADE,LockMode.UPGRADE_NOWAIT,LockMode.READ;

2. Hibernate 和 spring 中常出现的几个异常

org.springframework.beans.factory.BeanCreationException:

Error creating bean with name 'sessionFactory' defined in ServletContext resource
[/WEB-INF/applicationContext.xml]:

Initialization of bean failed; nested exception is org.hibernate.MappingException: Repeated column in
mapping for entity: com.xindeco.myregister.pojo.MyRegisterInfo column:

password (should be mapped with insert="false" update="false")

出错原因：password 和 repassword 同时对数据库表中的 password 一列，同时 update 和 insert
都设为 true。xml 文件如下：

```
<property name="password"
            type="java.lang.String"
            update="true"
            insert="true"
            access="property"
            column="password"
            length = "32"
        />

        <property name="repassword"
            type="java.lang.String"
            update="false"
            insert="false"
            access="property"
            column="password"
            length = "32"
        />
```

解决方法：将 repassword 的 insert 和 update 设为 false。

org.springframework.beans.factory.BeanCreationException:

Error creating bean with name 'sessionFactory' defined in ServletContext resource
[/WEB-INF/applicationContext.xml]:

Initialization of bean failed;nested exception is org.hibernate.PropertyNotFoundException: Could not
find a getter for ID in class

错误原因: hibernate 的映射文件中 ID 是大写的, 而 pojo 的类中 id 是小写的

解决方法: 要么将 pojo 类中的 id 改称大写, 要么把 hibernate 的映射文件中的 ID 改称小写。

3. Hibernate 与 jdbc 的联系

hibernate 是 jdbc 的轻量级封装, 包括 jdbc 的与数据库的连接 (用 hibernate.property 的配置文件实现当然本质是封装了 jdbc 的 forname),

和查询, 删除等代码, 都用面向对象的思想用代码联系起来, hibernate 通过 hbm 配置文件把 po 类的字段和数据库的字段关联起来比如数据库的 id,

在 po 类中就是 private Long id; public Long getId(); public setId(Long id);

然后 hql 语句也是面向对象的, 它的查询语句不是查询数据库而是查询类的, 这些实现的魔法就是 xml 文件, 其实 hibernate=封装的 jdbc+xml 文件

4. Hibernate 与 spring 的联系

hibernate 中的一些对象可以给 Spring 来管理, 让 Spring 容器来创建 hibernate 中一些对象实例化。例如: SessionFactory, HibernateTemplate 等。

Hibernate 本来是对数据库的一些操作, 放在 DAO 层, 而 Spring 给业务层的方法定义了事务, 业务层调用 DAO 层的方法, 很好的将 Hibernate 的操作也加入到事务中来了。

5. Hibernate 自带的分页机制是什么? 如果不使用 Hibernate 自带的分页, 则采用什么方式分页?

1. hibernate 自带的分页机制: 获得 Session 对象后, 从 Session 中获得 Query 对象。用 Query.setFirstResult(): 设置要显示的第一行数据,

Query.setMaxResults(): 设置要显示的最后一行数据。

2、不使用 hibernate 自带的分页, 可采用 sql 语句分页,

如: 5: 为每页显示的记录, 2 为当前页: select * top 5 from table where tabld not in (select tabld top (2-1)*5 from table);

6. hibernate 的对象的三种持久化状态, 并给出解释?

不清楚 hibernate 的对象的三种持久化状态, 只知道 hibernate 对象的三种状态, 下面有介绍。

7. hibernate 中一对多配置文件返回的是什么?

hibernate 中一对多配置文件会相应的映射为两张表, 并且它们之间的关系是一对多的。

例如: 一个 student 和 classes 表的关系。一个学生只能是一个班的, 一个班可以有多个学生。

8. update () 和 saveOrUpdate () 的区别?

update() 和 saveOrUpdate() 是用来对跨 Session 的 PO 进行状态管理的。

update() 方法操作的对象必须是持久化了的对象。也就是说, 如果此对象在数据库中不存在的话, 就不能使用 update() 方法。

saveOrUpdate() 方法操作的对象既可以使持久化了的, 也可以使没有持久化的对象。如果是持久化了的对象调用 saveOrUpdate() 则会

更新数据库中的对象; 如果是未持久化的对象使用此方法, 则 save 到数据库中。

9. hibernate 的三种状态之间如何转换

当对象由瞬时状态(Transient)一 save() 时, 就变成了持久化状态。

当我们在 Session 里存储对象的时候, 实际是在 Session 的 Map 里存了一份,

也就是它的缓存里放了一份, 然后, 又到数据库里存了一份, 在缓存里这一份叫持久对象(Persistent)。

Session 一 Close() 了, 它的缓存也都关闭了, 整个 Session 也就失效了,

这个时候，这个对象变成了游离状态(Detached)，但数据库中还是存在的。

当游离状态(Detached)update()时，又变为了持久状态(Persistent)。

当持久状态(Persistent)delete()时，又变为了瞬时状态(Transient)，

此时，数据库中并没有与之对应的记录。

10. hibernate 拒绝连接、服务器崩溃的原因？最少写 5 个

1. db 没有打开
2. 网络连接可能出了问题
3. 连接配置错了
4. 驱动的 driver，url 是否都写对了
5. LIB 下加入相应驱动，数据连接代码是否有误
6. 数据库配置可能有问题
7. 当前联接太多了，服务器都有访问人数限制的
8. 服务器的相应端口没有开，即它不提供相应的服务
9. hibernate 有哪些缓存，分别怎么使用？
10. 你对 hibernate 的了解到了一个什么样的程度？
11. 写出一个 sql 语句体现 hibernate 中一对多的关系

11. Hibernate 介绍

Hibernate 是一个开放源代码的对象关系映射框架，它对 JDBC 进行了非常轻量级的对象封装，使得 Java 程序员可以随心所欲的使用对象编程思维来操纵数据库。 Hibernate 可以应用在任何使用 JDBC 的场合，既可以在 Java 的客户端程序使用，也可以在 Servlet/JSP 的 Web 应用中使用，最具革命意义的是，Hibernate 可以在应用 EJB 的 J2EE 架构中取代 CMP，完成数据持久化的重任。

Hibernate 的核心接口一共有 5 个，分别为：Session、SessionFactory、Transaction、Query 和 Configuration。这 5 个核心接口在任何开发中都会用到。通过这些接口，不仅可以对持久化对象进行存取，还能够进行事务控制。下面对这五个核心接口分别加以介绍。

•**Session 接口**:Session 接口负责执行被持久化对象的 CRUD 操作(CRUD 的任务是完成与数据库的交流，包含了很多常见的 SQL 语句。)。但需要注意的是 Session 对象是非线程安全的。同时，Hibernate 的 session 不同于 JSP 应用中的 HttpSession。这里当使用 session 这个术语时，其实指的是 Hibernate 中的 session，而以后会将 HttpSession 对象称为用户 session。

•**SessionFactory 接口**:SessionFactory 接口负责初始化 Hibernate。它充当数据存储源的代理，并负责创建 Session 对象。这里用到了工厂模式。需要注意的是 SessionFactory 并不是轻量级的，因为一般情况下，一个项目通常只需要一个 SessionFactory 就够，当需要操作多个数据库时，可以为每个数据库指定一个 SessionFactory。

•**Configuration 接口**:Configuration 接口负责配置并启动 Hibernate，创建 SessionFactory 对象。在 Hibernate 的启动的过程中，Configuration 类的实例首先定位映射文档位置、读取配置，然后创建 SessionFactory 对象。

•**Transaction 接口**:Transaction 接口负责事务相关的操作。它是可选的，开发人员也可以设计编写自己的底层事务处理代码。

•**Query 和 Criteria 接口**:Query 和 Criteria 接口负责执行各种数据库查询。它可以使用 HQL 语言或 SQL 语句两种表达方式。

12. Hibernate 主键介绍

Assigned

Assigned 方式由程序生成主键值，并且要在 `save()` 之前指定否则会抛出异常

特点: 主键的生成值完全由用户决定，与底层数据库无关。用户需要维护主键值，在调用 `session.save()` 之前要指定主键值。

Hilo

Hilo 使用高低位算法生成主键，高低位算法使用一个高位值和一个低位值，然后把算法得到的两个值拼接起来作为数据库中的唯一主键。Hilo 方式需要额外的数据库表和字段提供高位值来源。默认情况下使用的表是

`hibernate_unique_key`，默认字段叫作 `next_hi`。`next_hi` 必须有一条记录否则会出现错误。

特点: 需要额外的数据库表的支持，能保证同一个数据库中主键的唯一性，但不能保证多个数据库之间主键的唯一性。Hilo 主键生成方式由 Hibernate 维护，所以 Hilo 方式与底层数据库无关，但不应该手动修改 `hi/lo` 算法使用的表的值，否则会引起主键重复的异常。

Increment

Increment 方式对主键值采取自动增长的方式生成新的主键值，但要求底层数据库的支持 `Sequence`。如 Oracle, DB2 等。需要在映射文件 `xxx.hbm.xml` 中加入 `Increment` 标志符的设置。

特点: 由 Hibernate 本身维护，适用于所有的数据库，不适合多进程并发更新数据库，适合单一进程访问数据库。不能用于群集环境。

Identity

Identity 当时根据底层数据库，来支持自动增长，不同的数据库用不同的主键增长方式。

特点: 与底层数据库有关，要求数据库支持 Identity，如 MySQL 中是 `auto_increment`，SQL Server 中是 `Identity`，支持的数据库有 MySQL、SQL Server、DB2、Sybase 和 HypersonicSQL。Identity 无需 Hibernate 和用户的干涉，使用较为方便，但不便于在不同的数据库之间移植程序。

Sequence

Sequence 需要底层数据库支持 `Sequence` 方式，例如 Oracle 数据库等

特点: 需要底层数据库的支持序列，支持序列的数据库有 DB2、PostgreSQL、Oracle、SAPDb 等在不同数据库之间移植程序，特别从支持序列的数据库移植到不支持序列的数据库需要修改配置文件

Native

Native 主键生成方式会根据不同的底层数据库自动选择 Identity、Sequence、Hilo 主键生成方式

特点: 根据不同的底层数据库采用不同的主键生成方式。由于 Hibernate 会根据底层数据库采用不同的映射方式，因此便于程序移植，项目中如果用到多个数据库时，可以使用这种方式。

UUID

UUID 使用 128 位 UUID 算法生成主键，能够保证网络环境下的主键唯一性，也就能够保证在不同数据库及不同服务器下主键的唯一性。

特点: 能够保证数据库中的主键唯一性，生成的主键占用比较大的存储空间

Foreign GUID

Foreign 用于一对一关系中。GUID 主键生成方式使用了一种特殊算法，保证生成主键的唯一性，支持 SQL Server 和 MySQL

13. Hibernate 源码中几个包的作用简要介绍

net.sf.hibernate.* 该包的类基本上都是接口类和异常类

net.sf.hibernate.cache.* JCS 的实现类

net.sf.hibernate.cfg.* 配置文件读取类

net.sf.hibernate.collection.* Hibernate 集合接口实现类，例如 List, Set, Bag 等等，Hibernate 之所以要自行编写集合接口实现类是为了支持 lazy loading

net.sf.hibernate.connection.* 几个数据库连接池的 Provider

net.sf.hibernate.dialect.* 支持多种数据库特性，每个 Dialect 实现类代表一种数据库，描述了该数据库支持的数据类型和其它特点，例如是否有 AutoIncrement，是否有 Sequence，是否有分页 sql 等等

net.sf.hibernate.eg.* Hibernate 文档中用到的例子

net.sf.hibernate.engine.* 这个包的类作用比较散

net.sf.hibernate.expression.* HQL 支持的表达式

net.sf.hibernate.hq.* HQL 实现

net.sf.hibernate.id.* ID 生成器

net.sf.hibernate.impl.* 最核心的包，一些重要接口的实现类，如果 Session, SessionFactory, Query 等

net.sf.hibernate.jca.* JCA 支持，把 Session 包装为支持 JCA 的接口实现类

net.sf.hibernate.jmx.* 我不懂 JMX，只知道 JMX 是用来编写 App Server 的管理程序的，大概是 JMX 部分接口的实现，使得 App Server 可以通过 JMX 接口管理 Hibernate

net.sf.hibernate.loader.* 也是很核心的包，主要是生成 sql 语句的

net.sf.hibernate.lob.* Blob 和 Clob 支持

net.sf.hibernate.mapping.* hbm 文件的属性实现

net.sf.hibernate.metadata.* PO 的 Meta 实现

net.sf.hibernate.odmg.* ODMG 是一个 ORM 标准，这个包是 ODMG 标准的实现类

net.sf.hibernate.persister.* 核心包，实现持久对象和表之间的映射

net.sf.hibernate.proxy.* Proxy 和 Lazy Loading 支持

net.sf.hibernate.ps.* 该包是 PreparedStatement Cache

net.sf.hibernate.sql.* 生成 JDBC sql 语句的包

net.sf.hibernate.test.* 测试类，你可以用 junit 来测试 Hibernate

net.sf.hibernate.tool.hbm2ddl.* 用 hbm 配置文件生成 DDL

net.sf.hibernate.transaction.* Hibernate Transaction 实现类

net.sf.hibernate.type.* Hibernate 中定义的持久对象的属性的数据类型

net.sf.hibernate.util.* 一些工具类，作用比较散

net.sf.hibernate.xml.* XML 数据绑定

14. 缓存管理

Hibernate 中提供了两级 Cache，第一级别的缓存是 Session 级别的缓存，它是属于事务范围的缓存。这一级别的缓存由 hibernate 管理的，一般情况下无需进行干预。第二级别的缓存是 SessionFactory 级别的缓存，它是属于进程范围或群集范围的缓存。这一级别的缓存可以进行配置和更改，并且可以动态加载和卸载。Hibernate 还为查询结果提供了一个查询缓存，它依赖于第二级缓存。

1. 一级缓存和二级缓存的比较：第一级缓存 第二级缓存 存放数据的形式 相互关联的持久化对象 对象的散装数据 缓存的范围 事务范围，每个事务都有单独的第一级缓存 进程范围或集群范围，缓存被同一个进程或集群范围内的所有事务共享 并发访问策略 由于每个事务都拥有单独的第一级缓存，不会出现并发问题，无需提供并发访问策略 由于多个事务会同时访问第二级缓存中相同数据，因此必须提供适当的并发访问策略，来保证特定的事务隔离级别 数据过期策略 没有提供 数据过期策略。处于一级缓存中的对象永远不会过期，除非应用程序显式清空缓存或者清除特定的对象 必须提供数据过期策略，如基于内存的缓存中的对象的最大数目，允许对象处于缓存中的最长时间，以及允许对象处于缓存中的最长空闲时间 物理存储介质 内存和硬盘。对象的散装数据首先存放在基于内存的缓存中，当内存中对象的数目达到数据过期策略中指定上限时，就会把其余的对象写入基于硬盘的缓存中。缓存的软件实现 在 Hibernate 的 Session 的实现中包含了缓存的实现 由第三方提供，Hibernate 仅提供了缓存适配器(CacheProvider)。用于把特定的缓存插件集成到 Hibernate 中。启用缓存的方式 只要应用程序通过 Session 接口来执行保存、更新、删除、加载和查询数据库数据的操作，Hibernate 就会启用第一级缓存，把数据库中的数据以对象的形式拷贝到缓存中，对于批量更新和批量删除操作，如果不希望启用第一级缓存，可以绕过 Hibernate API，直接通过 JDBC API 来执行指操作。用户可以在单个类或类的单个集合的粒度上配置第二级缓存。如果类的实例被经常读但很少被修改，就可以考虑使用第二级缓存。只有为某个类或集合配置了第二级缓存，Hibernate 在运行时才会把它的实例加入到第二级缓存中。 用户管理缓存的方式 第一级缓存的物理介质为内存，由于内存容量有限，必须通过恰当的检索策略和检索方式来限制加载对象的数目。Session 的 evict()方法可以显式清空缓存中特定对象，但这种方法不值得推荐。 第二级缓存的物理介质可以是内存和硬盘，因此第二级缓存可以存放大量的数据，数据过期策略的 maxElementsInMemory 属性值可以控制内存中的对象数目。管理第二级缓存主要包括两个方面：选择需要使用第二级缓存的持久类，设置合适的并发访问策略：选择缓存适配器，设置合适的数据过期策略。

2. 一级缓存的管理：当应用程序调用 Session 的 save()、update()、saveOrUpdate()、get() 或 load()，以及调用查询接口的 list()、iterate()或 filter()方法时，如果在 Session 缓存中还不存在相应的对象，Hibernate 就会把该对象加入到第一级缓存中。当清理缓存时，Hibernate 会根据缓存中对象的状态变化来同步更新数据库。Session 为应用程序提供了两个管理缓存的方法：evict(Object obj)：从缓存中清除参数指定的持久化对象。clear()：清空缓存中所有持久化对象。

3. 二级缓存的管理：

3.1. Hibernate 的二级缓存策略的一般过程如下：

1) 条件查询的时候，总是发出一条 select * from table_name where （选择所有字段）这样的 SQL 语句查询数据库，一次获得所有的数据对象。

2) 把获得的所有数据对象根据 ID 放入到第二级缓存中。

3) 当 Hibernate 根据 ID 访问数据对象的时候，首先从 Session 一级缓存中查；查不到，如果配置了二级缓存，那么从二级缓存中查；查不到，再查询数据库，把结果按照 ID 放入到缓存。

4) 删除、更新、增加数据的时候, 同时更新缓存。

Hibernate 的二级缓存策略, 是针对于 ID 查询的缓存策略, 对于条件查询则毫无作用。为此, Hibernate 提供了针对条件查询的 Query Cache。

3.2. 什么样的数据适合存放到第二级缓存中? 1 很少被修改的数据 2 不是很重要的数据,

允许出现偶尔并发的数据 3 不会被并发访问的数据 4 参考数据,指的是供应用参考的常量数据, 它的实例数目有限, 它的实例会被许多其他类的实例引用, 实例极少或者从来不会被修改。

3.3. 不适合存放到第二级缓存的数据? 1 经常被修改的数据 2 财务数据, 绝对不允许出现并发 3 与其他应用共享的数据。

3.4. 常用的缓存插件 Hibernate 的二级缓存是一个插件, 下面是几种常用的缓存插件:

EhCache: 可作为进程范围的缓存, 存放数据的物理介质可以是内存或硬盘, 对 Hibernate 的查询缓存提供了支持。

OSCache: 可作为进程范围的缓存, 存放数据的物理介质可以是内存或硬盘, 提供了丰富的缓存数据过期策略, 对 Hibernate 的查询缓存提供了支持。

SwarmCache: 可作为群集范围内的缓存, 但不支持 Hibernate 的查询缓存。

JBossCache: 可作为群集范围内的缓存, 支持事务型并发访问策略, 对 Hibernate 的查询缓存提供了支持。

3.5. 配置二级缓存的主要步骤:

1) 选择需要使用二级缓存的持久化类, 设置它的命名缓存的并发访问策略。这是最值得认真考虑的步骤。

2) 选择合适的缓存插件, 然后编辑该插件的配置文件。

Spring

1. 你一般用 spring 做什么?

控制反转 IOC AOP

2. spring 中的哪个类的哪个方法可用于获取 bean

3. spring 是什么? 根据你的理解详细谈谈你的见解。

◆目的: 解决企业应用开发的复杂性

◆功能: 使用基本的 JavaBean 代替 EJB, 并提供了更多的企业应用功能

◆范围: 任何 Java 应用

简单来说, Spring 是一个轻量级的控制反转(IoC)和面向切面(AOP)的容器框架。

◆轻量——从大小与开销两方面而言 Spring 都是轻量的。完整的 Spring 框架可以在一个大小只有 1MB 多的 JAR 文件里发布。并且 Spring 所需的处理开销也是微不足道的。此外, Spring 是非侵入式的: 典型地, Spring 应用中的对象不依赖于 Spring 的特定类。

◆控制反转——Spring 通过一种称作控制反转 (IoC) 的技术促进了松耦合。当应用了 IoC, 一个对象依赖的其它对象会通过被动的方式传递进来, 而不是这个对象自己创建或者查找依赖对象。你可以认为 IoC 与 JNDI 相反——不是对象从容器中查找依赖, 而是容器在对象初始化时不等对象请求就主动将依赖传递给它。

◆**面向切面**——Spring 提供了面向切面编程的丰富支持，允许通过分离应用的业务逻辑与系统级服务（例如审计（auditing）和事务（）管理）进行内聚性的开发。应用对象只实现它们应该做的——完成业务逻辑——仅此而已。它们并不负责（甚至是意识）其它的系统级关注点，例如日志或事务支持。

◆**容器**——Spring 包含并管理应用对象的配置和生命周期，在这个意义上它是一种容器，你可以配置你的每个 bean 如何被创建——基于一个可配置原型（prototype），你的 bean 可以创建一个单独的实例或者每次需要时都生成一个新的实例——以及它们是如何相互关联的。然而，Spring 不应该被混同于传统的重量级的 EJB 容器，它们经常是庞大与笨重的，难以使用。

◆**框架**——Spring 可以将简单的组件配置、组合成为复杂的应用。在 Spring 中，应用对象被声明式地组合，典型地是在一个 XML 文件里。Spring 也提供了很多基础功能（事务管理、持久化框架集成等等），将应用逻辑的开发留给了你。

所有 Spring 的这些特征使你能够编写更干净、更可管理、并且更易于测试的代码。它们也为 Spring 中的各种模块提供了基础支持。

4. 项目中如何体现 Spring 中的切面编程，距离说明。

面向切面编程：主要是横切一个关注点,将一个关注点模块化成一个切面。在切面上声明一个通知(Advice)和切入点(Pointcut); 通知：是指在切面的某个特定的连接点(代表一个方法的执行。通过声明一个 org.aspectj.lang.JoinPoint 类型的参数可以使通知（Advice）的主体部分获得连接点信息。)上执行的动作。通知中定义了要插入的方法。切入点：切入点的内容是一个表达式，以描述需要在哪些对象的哪些方法上插入通知中定义的方法。

项目中用到的 Spring 中的切面编程最多的地方：声明式事务管理。

- a、定义一个事务管理器
- b、配置事务特性(相当于声明通知。一般在业务层的类的一些方法上定义事务)
- c、配置哪些类的哪些方法需要配置事务(相当于切入点。一般是业务类的方法上)

5. spring 中可以用注入获得属性值，还有其他方式吗？

读取配置文件

6. spring 在项目中如何充当粘合剂

1、在项目中利用 spring 的 IOC(控制反转或依赖注入),明确地定义组件接口(如 UserDao), 开发者可以独立开发各个组件, 然后根据组件间的依赖关系组装(UserAction 依赖于 UserBiz, UserBiz 依赖于 UserDao)运行, 很好的把 Struts(Action)和 hibernate(DAO 的实现)结合起来了。

2、spring 的事务管理把 hibernate 对数据库的操作进行了事务配置。

7. spring 的事务如何配置

spring 的声明式事务配置：

1. <!-- 配置 sessionFactory -->

```
<bean id="sessionFactory"
      class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
    <property name="configLocation">
      <value>/WEB-INF/classes/hibernate.cfg.xml</value>
    </property>
  </bean>
```

2. 配置事务管理器

<!-- 配置事务管理器 -->

```
<bean id="transactionManager"
      class="org.springframework.orm.hibernate3.HibernateTransactionManager">
    <property name="sessionFactory">
      <ref local="sessionFactory" />
    </property>
</bean>
```

3. 配置事务特性

```
<tx:advice id="txAdvice" transaction-manager="transactionManager">
  <tx:attributes>
    <tx:method name="add*" propagation="REQUIRED"/>
    <tx:method name="update*" propagation="REQUIRED"/>
    <tx:method name="del*" propagation="REQUIRED"/>
    <tx:method name="*" read-only="true"/>
  </tx:attributes>
</tx:advice>
```

4. 配置哪些类的哪些方法配置事务

```
<aop:config>
  <aop:pointcut id="allManagerMethod" expression="execution(* com.yyaccp.service.impl.*(..))"/>
  <aop:advisor advice-ref="txAdvice" pointcut-ref="allManagerMethod">
</aop:config>
```

isolation 设定事务的隔离级别,事务管理器根据它来控制另外一个事务可以看到本事务内的哪些数据。

定义的 5 个不同的事务隔离级别:

DEFAULT: 默认的隔离级别, 使用数据库默认的事务隔离级别

READ_COMMITTED: 保证一个事务修改的数据提交后才能被另外一个事务读取。另外一个事务不能读取该事务未提交的数据。这种事务隔离级别可以避免脏读出现, 但是可能会出现不可重复读和幻像读。

READ_UNCOMMITTED: 这是事务最低的隔离级别, 它允许另外一个事务可以看到这个事务未提交的数据。这种隔离级别会产生脏读, 不可重复读和幻像读。

REPEATABLE_READ: 这种事务隔离级别可以防止脏读, 不可重复读。但是可能出现幻像读。它除了保证一个事务不能读取另一个事务未提交的数据外, 还保证了避免不可重复读。

SERIALIZABLE: 这是花费最高代价但是最可靠的事务隔离级别。事务被处理为顺序执行。除了防止脏读, 不可重复读外, 还避免了幻像读。

propagation 定义了 7 个事务传播行为

REQUIRED: 如果存在一个事务, 则支持当前事务。如果没有事务则开启一个新的事务。

SUPPORTS: 如果存在一个事务, 支持当前事务。如果没有事务, 则非事务的执行。但是对于事务同步的事务管理器, SUPPORTS 与不使用事务有少许不同。

REQUIRES_NEW 总是开启一个新的事务。如果一个事务已经存在, 则将这个存在的事务挂起。

NOT_SUPPORTED 总是非事务地执行, 并挂起任何存在的事务。

NEVER 总是非事务地执行, 如果存在一个活动事务, 则抛出异常

NESTED: 如果一个活动的事务存在, 则运行在一个嵌套的事务中。如果没有活动事务, 则按

TransactionDefinition.PROPROPAGATION_REQUIRED 属性执行。

嵌套事务一个非常重要的概念就是内层事务依赖于外层事务。外层事务失败时, 会回滚内层事务所做的动作。而内层事务操作失败并不会引起外层事务的回滚。

REQUIRED 应该是我们首先的事务传播行为。它能够满足我们大多数的事务需求。

8. transaction 有那几种实现(事务处理)(Spring)

在 Spring 中，事务处理主要有两种方式

(1) 代码控制事务

在程序中引入新的模版类，这个类封装了事务管理的功能

(2) 参数配置控制事务,在 Application-Context.xml 增加一个事务代理（UserDAOProxy）配置

2. IBATIS 中的事物属性怎么配置

SQL MAP XML 配置文件中的 transactionManager